

MindEase — Complete Project Analysis

Project: MindEase — AI-powered Stress Monitoring Android Application

Tech Stack: Kotlin (Jetpack Compose) + Python (FastAPI) + MongoDB + ML (Random Forest) + Gemini AI

1. All Equations Used in the Project

Equation 1 — Eye Aspect Ratio (EAR) for Blink Detection

File: `blink_api/app.py`

Used to detect eye blinks from video frames. MediaPipe extracts 6 landmark points per eye:

$$EAR = (A + B) / (2.0 \times C)$$

Where: - **A** = $\|P_2 - P_6\|$ (vertical distance between upper-left and lower-left eyelid) - **B** = $\|P_3 - P_5\|$ (vertical distance between upper-right and lower-right eyelid) - **C** = $\|P_1 - P_4\|$ (horizontal distance between eye corners) - $\| \|$ = Euclidean norm: $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

⚠ IMPORTANT:

Threshold: $EAR < 0.21 \rightarrow$ Eye is closed

Blink counted when EAR stays below threshold for ≥ 2 consecutive frames

The average EAR of both eyes is used:

$$EAR_{avg} = (EAR_{left} + EAR_{right}) / 2.0$$

Equation 2 — Blink Rate (Blinks Per Minute)

File: `CameraScanScreen.kt`

ML Kit detects blinks in a **15-second window**, then extrapolates to per-minute rate:

$$Blink_Rate_BPM = blink_count_in_15s \times 4$$

i NOTE:

Eye closed threshold: `eyeOpenProbability < 0.2`

Eye open threshold: `eyeOpenProbability > 0.5`

Equation 3 — Accelerometer Magnitude (Step Detection & Motion)

File: `StepManager.kt` and `MotionSensorManager.kt`

```
magnitude =  $\sqrt{x^2 + y^2 + z^2}$ 
```

Where `x`, `y`, `z` are the 3-axis accelerometer readings.

For step detection (`StepManager`):

```
IF magnitude > 11.5 AND lastMagnitude ≤ 11.5 AND (currentTime - lastStepTime) > 300ms  
THEN step_count++
```

For motion/stillness detection (`MotionSensorManager`):

```
deviation = |magnitude - 9.81|  
  
IF deviation > 0.5 → motion detected (not still)  
IF deviation ≤ 0.5 for ≥ 20 minutes → declared "still"
```

Equation 4 — Heart Rate Estimation (Peak Detection via PPG)

File: `CameraScanScreen.kt`

Uses **photoplethysmography (PPG)** — finger over rear camera with flashlight:

```
Step 1: avg_brightness =  $\Sigma(\text{pixel\_values}) / \text{num\_pixels}$  (per frame)  
Step 2: smoothed = moving_average(signal, window=5)  
Step 3: mean = average(smoothed)  
Step 4: peaks = count(transitions from below_mean → above_mean)  
Step 5: duration_seconds = num_frames / 30.0 (assuming 30 fps)  
Step 6: BPM = (peaks / duration_seconds) × 60
```

Final HR uses **median** of last 15 readings for robustness:

```
sorted_readings = sort(heartRateBuffer)  
IF even count: median = (sorted[n/2 - 1] + sorted[n/2]) / 2  
IF odd count: median = sorted[n/2]
```

Equation 5 — Sleep Confidence Score (Weighted Multi-Signal)

File: `SleepDetectionManager.kt`

```
Confidence = W_screen + W_motion + W_window + W_no_unlock
```

Signal	Weight	Full Score Condition	2/3 Score	1/3 Score
Screen Off	30	≥ 30 min off	≥ 15 min	≥ 10 min

Signal	Weight	Full Score Condition	2/3 Score	1/3 Score
Motion Still	30	≥ 30 min still	≥ 20 min	≥ 10 min
Sleep Window	25	Currently in predicted window	—	—
No Unlock	15	Last unlock > 15 min ago	—	—

Penalty: If not still but screen off and motion magnitude > 1.0 → score -10

```
Final Confidence = clamp(score, 0, 100)
```

State transitions: - AWAKE → POSSIBLY_SLEEPING: confidence ≥ **40** + screen off ≥ 15 min - POSSIBLY_SLEEPING → SLEEPING: confidence ≥ **60** + screen off ≥ 30 min + still

Equation 6 — Circular Mean for Sleep Window Prediction

File: `SleepPatternRepository.kt`

Handles midnight wraparound (e.g., averaging 23:00 and 01:00):

```
For each hour h:
    θ = h × 2π / 24

sinSum = Σ sin(θi) / n
cosSum = Σ cos(θi) / n

mean_angle = atan2(sinSum, cosSum)
IF mean_angle < 0: mean_angle += 2π

mean_hour = mean_angle × 24 / (2π)
```

Equation 7 — Sleep Duration Calculation

File: `SleepDetectionManager.kt`

```
duration_ms = wakeTime - sleepStartTime
duration_hours = duration_ms / (1000 × 60 × 60)
```

Clamped to range `[0.0, 16.0]` hours. Only saved if ≥ 1.0 hour.

Equation 8 — Estimated Steps (Fallback)

File: `HealthConnectManager.kt`

```
waking_hours = current_hour - 7      (for hours 7-22)
estimated_steps = waking_hours × 350
```

Equation 9 — Estimated Sleep from Time (Last Resort Fallback)

File: HealthConnectManager.kt

```
IF hour in [5..11]: estimated_sleep = clamp(hour - 23 + 24, 5.0, 8.0)
IF hour in [12..23]: estimated_sleep = 7.0
ELSE:
    estimated_sleep = 4.0
```

Equation 10 — ML Model: StandardScaler Transform

File: retrain_model.py

```
X_scaled = (X - μ) / σ
```

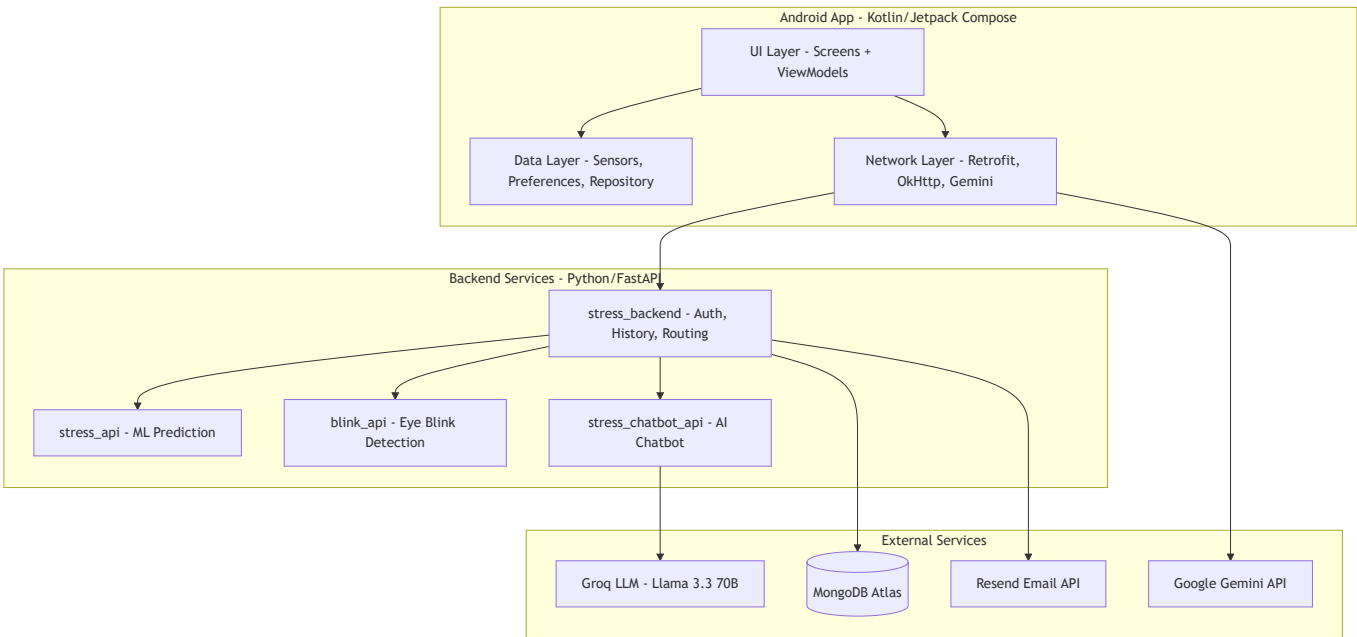
Where μ and σ are per-feature mean and standard deviation from training data.

Equation 11 — Stress Level from Score (Dashboard Fallback)

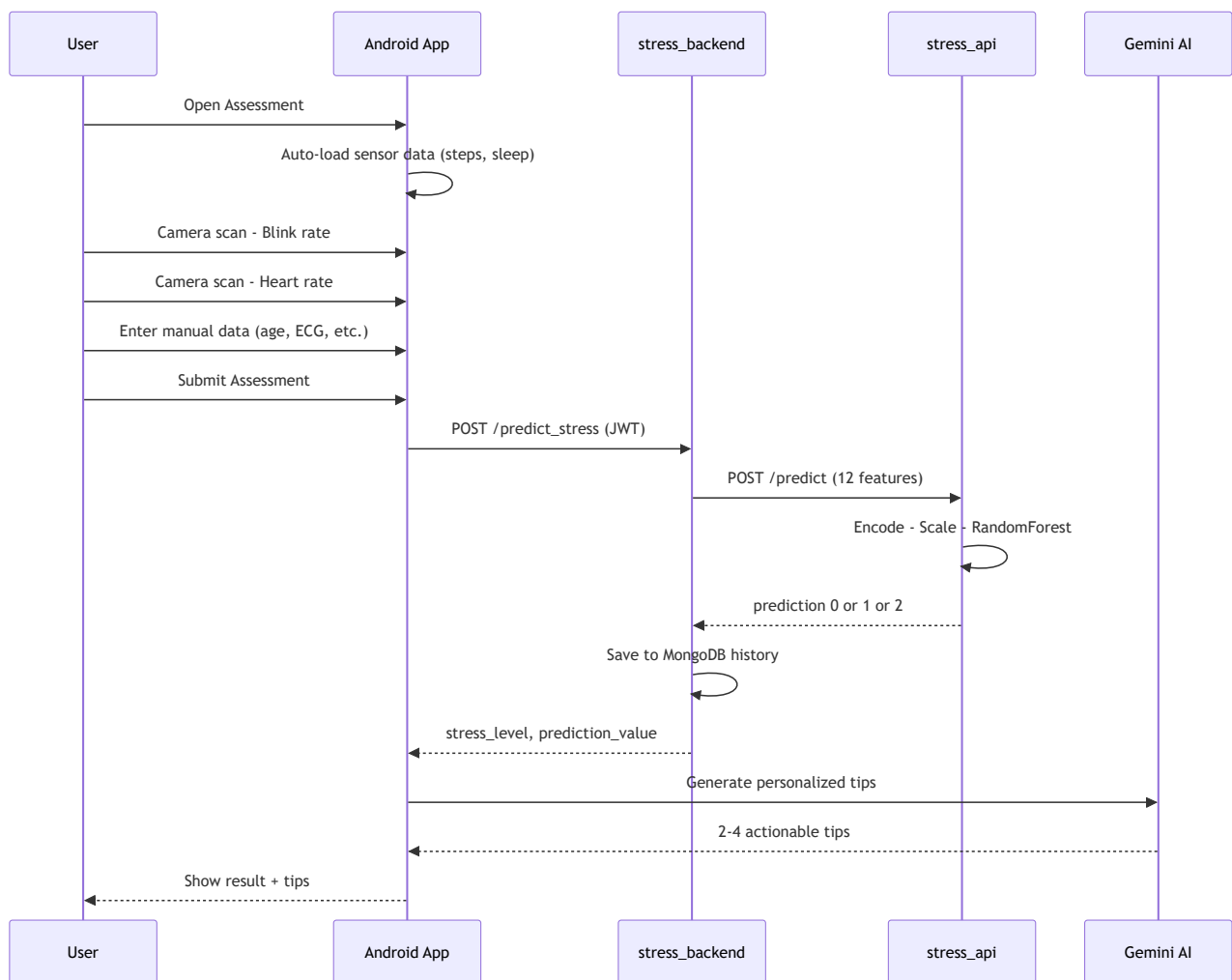
File: DashboardViewModel.kt

```
IF stress_score < 40 → "Low"
IF stress_score < 70 → "Mid"
ELSE
    → "High"
```

2. Project Architecture



3. Data Flow — Stress Prediction Pipeline



4. ML Model Features (12 Input Features)

#	Feature Name	Type	Range / Values	Source
1	Eye_Blink_Rate_per_min	Numeric	10–30 blinks/min	Camera (ML Kit / MediaPipe)
2	Resting_Heart_Rate_BPM	Numeric	45–180 BPM	Camera (PPG via rear lens)
3	Heart_Rate_After_Exercise_BPM	Numeric	60–200 BPM	Camera (PPG post-exercise)
4	age	Numeric	18–80 years	User input
5	gender	Categorical	Male, Female	User input
6	worklife	Categorical	Student, Employee, etc.	User input
7	Sleep_Duration_Hours	Numeric	0–16 hours	Auto-detected / manual
8	Chest_Pain	Categorical	No, Mild, Yes	User input
9	Cholesterol_mg_dL	Numeric	100–350 mg/dL	User input

#	Feature Name	Type	Range / Values	Source
10	ECG_Result	Categorical	Normal, Sinus_Tachycardia	User input
11	Body_Temperature_C	Numeric	35–40 °C	User input
12	Steps_per_Day	Numeric	0–30000 steps	Auto-tracked / manual

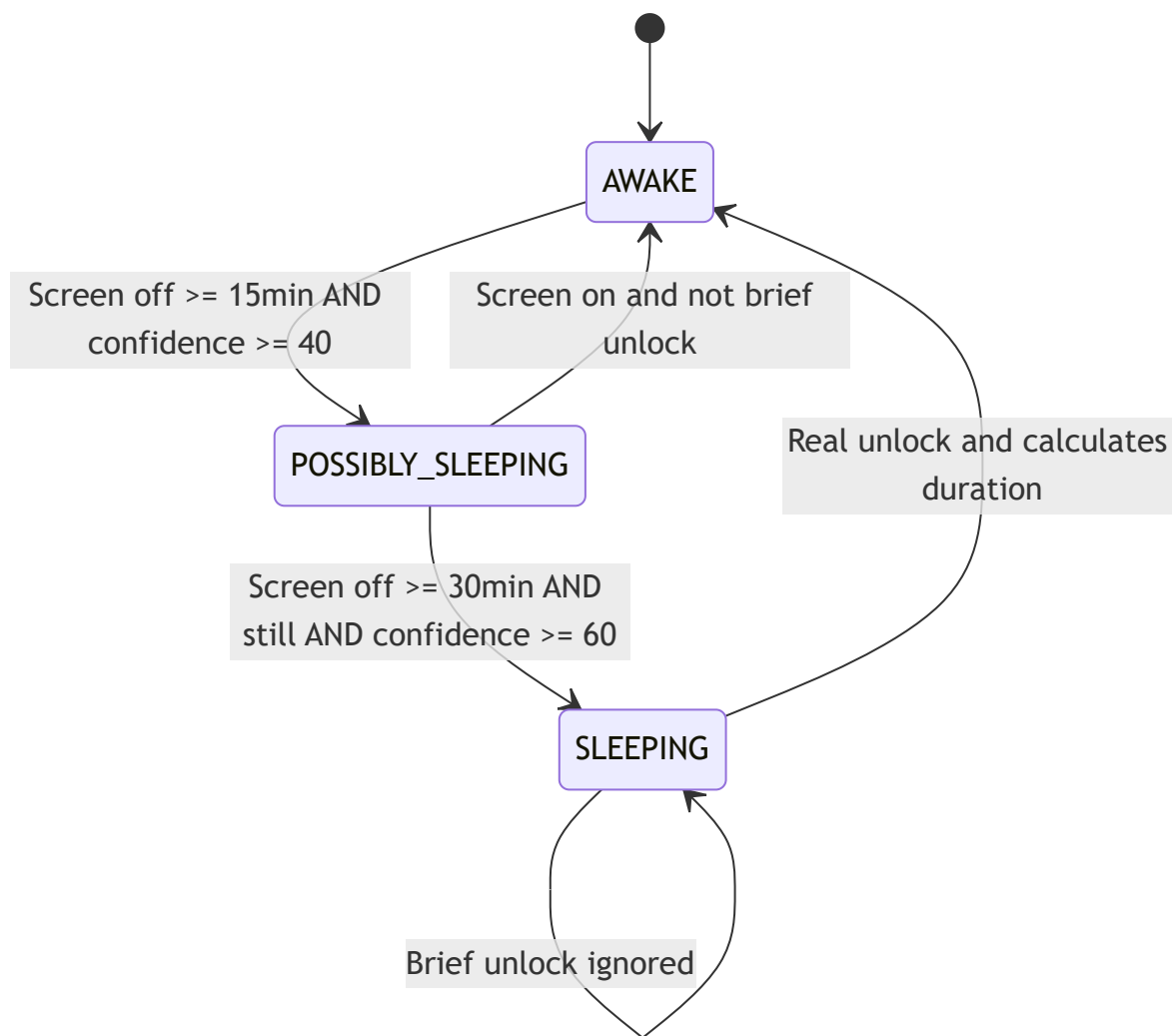
5. ML Model Configuration

Parameter	Value
Algorithm	Random Forest Classifier
n_estimators	200
max_depth	15
min_samples_split	5
min_samples_leaf	2
class_weight	balanced
Test split	80/20 (stratified)
Scaler	StandardScaler
Encoder	LabelEncoder (for categoricals)
Target	Stress_Level (Low=0, Medium=1, High=2)
Dataset	2000 samples

6. Stress Classification Output

Prediction Value	Stress Level	Color in App	Interpretation
0	Low	Green	Healthy stress levels
1	Medium	Yellow/Orange	Moderate stress, needs attention
2	High	Red	High stress, action recommended

7. Sleep Detection State Machine



8. Sleep Detection Thresholds

Parameter	Value	Purpose
Evaluation Interval	60 seconds	Battery-efficient loop
Screen Off Min	15 minutes	Trigger POSSIBLY_SLEEPING
Screen Off Strong	30 minutes	Strong sleep signal
Stillness Min	20 minutes	Motion-based confirmation
Brief Unlock	< 2 minutes	Ignored during sleep
Valid Sleep Duration	1.0 – 16.0 hours	Sanity filter
Motion Spike Threshold	magnitude > 1.0	Wake detection
Stillness Threshold	deviation ≤ 0.5 from gravity	Accelerometer stillness

Parameter	Value	Purpose
Rolling Window	60 samples (~5 min)	Motion averaging
Sensor Batch Latency	5 seconds	Battery optimization
Historical Sessions	Last 14 days	Pattern learning

9. API Endpoints

Method	Endpoint	Service	Auth	Description
GET	/	stress_backend	No	Health check
POST	/signup	stress_backend	No	User registration with OTP
POST	/verify_email	stress_backend	No	OTP verification
POST	/login	stress_backend	No	Login → JWT token
POST	/predict_stress	stress_backend	JWT	Submit assessment → stress prediction
GET	/history/{user_id}	stress_backend	JWT	Fetch scan history
POST	/chat	stress_backend	No	Chatbot proxy
POST	/change_password	stress_backend	JWT	Change user password
POST	/predict	stress_api	No	Raw ML prediction (internal)
POST	/blink-count	blink_api	No	Video → blink count (MediaPipe)
POST	/chat	stress_chatbot_api	No	LLM chatbot (Groq/Llama 3.3)

10. Authentication Flow

Component	Technology
Password Hashing	bcrypt
Token Format	JWT (HS256)
Token Expiry	30 days (43200 min)
Email OTP	6-digit random code
Primary Email	Resend API
Fallback Email	Gmail SMTP (SSL, port 465)
Fallback (no email)	Direct account creation

11. Sensor Data Priority Hierarchy

Steps:

- 1. Hardware Step Counter sensor (TYPE_STEP_COUNTER)
- 2. Hardware Step Detector sensor (TYPE_STEP_DETECTOR)
- 3. Accelerometer-based step detection (magnitude threshold)
- 4. Background service daily count
- 5. Health Connect API
- 6. Time-based estimation (350 × waking_hours)

Sleep:

- 1. SleepDetectionManager (real-time multi-signal engine)
- 2. SleepPatternRepository (last 24h cached session)
- 3. Health Connect SleepSessionRecord
- 4. UsageStats longest inactive gap
- 5. Time-of-day estimation (5-8 hours)

12. App Screens & Navigation

Screen	Purpose	Key Features
Welcome	Onboarding	Animated intro slides
Login	Authentication	Email + password, JWT storage
Signup	Registration	Name, email, password + OTP verification
Verify Email	OTP Entry	6-digit code verification
Dashboard	Home screen	Live steps, sleep, last stress level, streak
Assessment	Stress scan	6-step wizard (info → blink → HR → sleep → steps → submit)
Camera Scan	Blink/HR	ML Kit face detection + PPG heart rate
History	Past scans	Timeline view with stress trends
Profile	User info	Edit name, change password, logout
Chatbot	AI assistant	Medical stress chatbot (Groq LLM)
Doctors Map	Find doctors	Google Maps integration
Wellness Tips	Health tips	General wellness advice

13. Chatbot Safety Filters

Filter	Keywords	Action
Emergency	severe chest pain, can't breathe, fainting, suicidal, heart attack	Immediate emergency response
Medical	stress, heart, chest, sleep, anxiety, ecg, cholesterol, etc.	Allow — process with LLM
Off-topic	Anything else	Block — restricted response

LLM Setting	Value
Model	Llama 3.3 70B Versatile
Provider	Groq Cloud
Temperature	0.4
System Prompt	Medical stress advisor with safety guardrails

14. External Deployment URLs

Service	Hosting	URL
stress_backend	Render	stress-backend-*.onrender.com
stress_api	Render	stress-api-99xw.onrender.com
blink_api	Render	blink-api-jxdv.onrender.com
stress_chatbot_api	Render	stress-chatbot-api.onrender.com
Database	MongoDB Atlas	Cloud cluster (stress_db)

15. Database Schema

Database: stress_db (MongoDB)

Collection	Fields	Purpose
users	_id, name, email, password (bcrypt hash)	Registered users
pending_users	name, email, password, otp_code, created_at	OTP verification queue
history	user_id, input (StressInput), prediction_value, stress_level, timestamp	Scan results

16. File Structure Summary

Component	Files	Language	Lines of Code (approx)
Android UI Screens	13 files	Kotlin	~3,600
Android Data Layer	10 files	Kotlin	~1,500
Android Network	5 files	Kotlin	~700
stress_backend	3 files	Python	~420
stress_api	4 files	Python	~290
blink_api	1 file	Python	~83
stress_chatbot_api	1 file	Python	~92
Total	~37 files	Kotlin + Python	~6,700

17. Gemini AI Integration

Setting	Value
Model	gemini-2.0-flash
Temperature	0.7
Max Output Tokens	400
Output	2–4 personalized health tips
Fallback	Rule-based tips from user data
Trigger Conditions	Sleep < 6h, Steps < 3000, HR > 100, Chest pain, Cholesterol > 200, Blink > 20

18. Complete Data Flow Graph

